

02 — Build a two-layer ocean and test an effective carbon pump

In 01, adding the inferred TA allowed a uniform ocean to recover the reference state. Here you divide that ocean into surface and deep layers, then introduce a downward carbon transfer to maintain a DIC difference between them.

Follow three steps: **divide the ocean** → **maintain a gradient** → **supply the carbon needed for the full reference state**.

Learning goals: check that adding a reservoir conserves carbon and TA; explain a maintained DIC gradient as a balance between pump and mixing; calculate a finite carbon input and verify its inventory.

Provisional time: 55 minutes. Follow the same predict → map → run → check → explain sequence as in 01.

Part	Main question	Time
A. Add the deep box	Does the two-layer model recover the buffered 01 equilibrium without a pump?	20 min
B. Add the effective pump	What pump coefficient maintains the reference DIC ratio?	20 min
C. Add carbon	How much carbon allows the pumped model to reach the full reference state?	15 min

Complete the marked scientific choices and the two derivations. Constructor syntax, chemistry, plots, restarts and budget checks are supplied. [Teaching goals](#).

Optional coding reference: [From conceptual model to code \(two-page handout\)](#). Essential syntax is explained below. Code labels show where to focus: **Choose and explain** = complete the marked scientific choices; **Understand and run** = trace the supplied model and read its evidence; **Supplied implementation** = run the supporting machinery as provided. This practical is ungraded. Syntax memorisation is not required: refer to the examples and focus on connecting scientific assumptions to the code.

Reading key: **Key term** = concept to notice; **Question** = student prompt. Instructor answers use labelled purple panels in the instructor sheet. These reading cues complement the code labels above.

LECTURER REVIEW | INSTRUCTOR EDITION

Executed in ESBMTK314. All instructor solutions, code, execution outputs and optional disclosures are included. No Jupyter installation is needed to read this PDF.

Source: 02_two_layer_ocean_carbon_pump.ipynb

Execution: 2026-09-24. The source notebooks are unchanged.

```
# Supplied implementation: run this support code as provided.
from pathlib import Path
import sys
import numpy as np
import matplotlib.pyplot as plt
import PyCO2SYS as pyco2

ROOT = next(p for p in (Path.cwd(), *Path.cwd().parents)
             if (p / 'teaching_config.py').is_file())
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
from teaching_config import TEACHING as config
from simple_models import (
    new_model, box_parameters, box_mass_kg, connect_atmosphere,
    single_box, inventories, audit, mixing_mass_transport,
    finite_pulse_clock, integrated_signal,
)
from esbmtk import (
```

```

GasReservoir, Species2Species, Signal, Source,
initialize_reservoirs, create_bulk_connections, add_carbonate_system_1,
)
from model import run_model
from teaching_plots import plot_pump_comparison, plot_synthetic_forcing

```

A. Add a deep box and check the unpumped model

A1. Keep the 01 inventory and chemistry

```

Atmosphere -- J_gas,in --> Surface -- J_mix,down --> Deep
Atmosphere <-- J_gas,out -- Surface <-- J_mix,up ----> Deep
                CO2 only                DIC and TA
                Surface -- J_pump -----> Deep (Part B only)

```

Only the surface exchanges CO₂ with the atmosphere. The surface and deep volumes together equal the ocean volume in 01. Both layers use the inferred 01 TA, T = 16 °C, S = 35 and P = 0 bar, with the same carbonate settings. The combined atmosphere-plus-ocean carbon inventory also stays the same. Real thermal and TA gradients are omitted here; 03/04 use distinct box conditions.

Supplied geometry. The layer split in `teaching_config.py` is calculated algebraically before any model run. It makes the reference DIC values (surface 2040, deep 2250 μmol/kg) give an ocean/atmosphere carbon inventory ratio $r_{\text{ocn/atm}} = C_{\text{ocn}}/C_{\text{atm}} = 62.4$ at 280 ppm, where ocn and atm denote ocean and atmosphere. Using the independent whole-ocean volume and atmospheric size, with ESBMTK density, gives a surface depth of about **298.75 m**. This is **ratio-derived teaching geometry**, rather than an observed mixed-layer depth. You will use the ratio again in Part C; preparing this geometry does not add carbon.

A2. Translate the mixing arrows into fluxes

As in 01, write each budget as **inputs minus outputs**. The two mixing arrows are equal water transports Q , each carrying its **source concentration**. Subscripts s and d denote surface and deep water. For $X = \text{DIC or TA}$,

$$J_{\text{mix,down}}^{(X)}(t) = Q\rho X_s(t), \quad J_{\text{mix,up}}^{(X)}(t) = Q\rho X_d(t).$$

The prescribed transport is 20 Sv in each direction (1 Sv = 10^6 m³/s). Use Q in m³/yr, ρ in kg/m³, and concentrations in mol/kg (TA in equivalents/kg): these give mol C/yr or TA equivalents/yr. Each arrow enters one box and leaves the other. Equal water flows preserve both box volumes. The supplied `create_bulk_connections` call constructs each direction for both tracers. Air–sea exchange retains invasion and outgassing from 01, evaluated together by one native gas connection.

Why keep TA transport? Both layers start with the same TA, and none of the processes represented here creates a TA difference. The opposing TA fluxes therefore cancel: mixing causes no net TA redistribution. We retain its transport for a consistent water-transfer model while isolating the DIC pump. TA still controls carbonate chemistry; it is the spatial TA gradient that we omit.

A3. Complete the reservoir and mixing choices

`build_layers` constructs a fresh model and returns it without running it. Both layers initially have 1000 μmol/kg DIC, the alternative partition in 01, and the inferred 01 TA. The atmosphere helper assigns the remaining carbon to the atmosphere to preserve the total inventory.

Reading this code.

Pattern	Meaning
<code>config.surface_volume_m3</code>	Retrieve a named setting from the shared configuration; here, surface volume in m ³ .
<code>state['Surface']</code>	Retrieve the surface entry from the state dictionary. Keys are case-sensitive: 'Surface' and 'Deep'.
<code>(1000.0, inferred_ta)</code>	A pair of concentrations in the order DIC, TA , both in $\mu\text{mol/kg}$.
<code>surface_dic, surface_ta = state['Surface']</code>	Unpack the pair into two variables.
<code>*state['Surface']</code>	Pass the two values as separate arguments to <code>box_parameters</code> .
<code>state=None</code>	Use the default initial values when no restart state is provided.

`create_bulk_connections` builds repeated routes for the species listed in `sp`. Each route key has the format **Source_to_Sink@id**: ESBMTK uses the two box names to find its endpoints, and `id` to identify the connection. The names must match the created reservoirs. The `@` belongs to ESBMTK's string format; it is not a separate Python operation. These IDs can distinguish connections in summaries or later lookups; 02 does not look them up directly.

Choose a flux law. The bulk dictionary's `ty` and an individual connection's `ctype` select the same kind of law. Choose from these options by matching the equation; the remaining constructor arguments are supplied for this exercise.

Choice	Flux law	Relevant arguments
regular	A prescribed flux independent of source concentration.	rate (bulk key <code>ra</code>), in amount/time.
scale_with_concentration	A coefficient multiplied by source concentration.	scale (bulk key <code>sc</code>); units must convert concentration to amount/time.
gasexchange	Net invasion minus outgassing, as in 01.	Gas-transfer and solubility settings, supplied by the atmosphere helper.

Question — map reservoirs and mixing

Complete **02.1** (deep-box volume and initial concentrations) and **02.2** (the two directed arrow names, transported species and `mixing_type`). Use IDs `mix_down` and `mix_up`. Choose one law for both mixing directions and explain how it implements the equation in A2. The surface construction is your example.

Leave **02.3** for Part B: its pump block is skipped while `k_kg_yr=0`. Run the definition cell, then the supplied comparison.

```
# Choose and explain: complete marked choices; surrounding machinery is supplied.
inferred_ta = float(config.reference_state()['alkalinity'])
print('Ratio-derived surface depth (m):', config.surface_depth_m)
print('Volumes (m3):', config.surface_volume_m3, config.deep_volume_m3)
assert config.surface_volume_m3 + config.deep_volume_m3 == config.ocean_volume_m3

def build_layers(k_kg_yr=0.0, state=None, *,
                stop='30 kyr', max_timestep='20 yr'):
    M = new_model(stop=stop, max_timestep=max_timestep)
    if state is None:
        state = {'Surface': (1000.0, inferred_ta), 'Deep': (1000.0, inferred_ta)}
    # Exercise 02.1: assign deep_volume_m3 and the two deep initial concentrations.
    # Geometry is supplied in config; state stores (DIC, TA) in  $\mu\text{mol/kg}$  for each box.
    # BEGIN SOLUTION
    deep_volume_m3 = config.deep_volume_m3
    deep_dic_umol_kg, deep_ta_umol_kg = state['Deep']
    # END SOLUTION
```

```

initialize_reservoirs(M, {
    'Surface': box_parameters(M, config.surface_volume_m3, *state['Surface']),
    'Deep': box_parameters(M, deep_volume_m3, deep_dic_umol_kg, deep_ta_umol_kg),
})
add_carbonate_system_1([M.Surface, M.Deep])
# Supplied: Q in m3/yr times ESBMTK rho in kg/m3 gives kg/yr.
transport_kg_yr = mixing_mass_transport()
# Exercise 02.2: choose both arrows, transported_species and mixing_type.
# Names follow 'Source_to_Sink@id'; use IDs mix_down and mix_up.
# BEGIN SOLUTION
down_arrow = 'Surface_to_Deep@mix_down'
up_arrow = 'Deep_to_Surface@mix_up'
transported_species = [M.DIC, M.TA]
mixing_type = 'scale_with_concentration'
# END SOLUTION
create_bulk_connections({
    down_arrow: {'ty': mixing_type, 'sc': transport_kg_yr,
                 'sp': transported_species},
    up_arrow: {'ty': mixing_type, 'sc': transport_kg_yr,
               'sp': transported_species},
}, M)
if k_kg_yr:
    # Exercise 02.3 (Part B): choose pump_source, pump_sink, pump_type and pump_scale.
    # This arrow transfers DIC only; its scale is the fitted kg/yr coefficient.
    # Leave this block for Part B: it is not used by the no-pump run.
    # BEGIN SOLUTION
    pump_source = M.Surface.DIC
    pump_sink = M.Deep.DIC
    pump_scale = k_kg_yr
    pump_type = 'scale_with_concentration'
    # END SOLUTION
    Species2Species(source=pump_source, sink=pump_sink,
                     ctype=pump_type, scale=float(pump_scale),
                     id='effective_pump')
connect_atmosphere(M, [M.Surface, M.Deep])
return M

```

Ratio-derived surface depth (m): 298.75014096166706

Volumes (m3): 1.0755005074620014e+17 1.2424499492538e+18

Run the supplied no-pump comparison

The next cell runs `build_layers()` with the pump off and the corresponding buffered one-box model from 01. `audit` checks carbon and TA conservation; `assert_allclose` checks that ocean masses and final concentrations agree within numerical tolerances. A failed assertion stops the cell so you can inspect the relevant choices before continuing.

```

# Understand and run: trace this supplied step and interpret its evidence.
# Supplied structural checks: compare with 01 before introducing the pump.
pump_off = build_layers()
one_box = single_box(ta_umol_kg=inferred_ta, initial_dic_umol_kg=1000)
for case in (one_box, pump_off):
    run_model(case)
    audit(case)
np.testing.assert_allclose(sum(box_mass_kg(b) for b in pump_off.ocean_boxes),
                           box_mass_kg(one_box.Ocean), rtol=1e-12)
np.testing.assert_allclose(pump_off.Surface.DIC.c[-1], pump_off.Deep.DIC.c[-1], atol=0.2e-6)
np.testing.assert_allclose(pump_off.Surface.DIC.c[-1], one_box.Ocean.DIC.c[-1], atol=0.2e-6)
np.testing.assert_allclose(pump_off.CO2_At.c[-1], one_box.CO2_At.c[-1], atol=0.5e-6)
print('Checks passed: carbon/TA conserved; no-pump equilibrium agrees with 01.')

```

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G. Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it
under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G. Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

status=0

message=The solver successfully reached the end of the integration interval.

Execution took 0.59 CPU seconds, wall time = 0.59 seconds

status=0

message=The solver successfully reached the end of the integration interval.

Execution took 0.67 CPU seconds, wall time = 0.66 seconds

=====

There were 89 warnings, check M.log

=====

Checks passed: carbon/TA conserved; no-pump equilibrium agrees with 01.

A4. Explain the structural check

A **stationary state** has no further change in any reservoir inventory, even though individual transfers continue. Stars denote stationary concentrations: DIC_s^* and DIC_d^* .

Question — explain the structural check

Without a pump, why must surface and deep DIC become equal at equilibrium? Why should this endpoint agree with the buffered one-box model? What would non-conservation tell you about your connections?

Instructor answer

With no pump, deep accumulation stops only when upward and downward carbon fluxes balance, so $DIC_d^* = DIC_s^*$. With the same total ocean mass, carbon, TA and chemistry, the uniform ocean has the same equilibrium partition as 01. Equal water transports preserve box volumes. Each internal DIC/TA transfer must leave one box and enter the other with the same flux; a budget drift signals a coding or unit problem.

B. Derive and test the effective pump

B1. Read the supplied pump assumption

Assumption: effective downward export is **first order in surface DIC**:

$$J_{pump}(t) = kDIC_s(t).$$

Here, **first order** means that the flux is proportional to the current surface DIC: doubling that concentration doubles the flux at fixed k . This **effective pump** transfers DIC directly from surface to deep water, without water or TA transfer. It represents the combined effect of unresolved processes; it has no explicit particles or organisms. The coefficient k stays constant here. Treat proportionality to the whole DIC pool as a supplied model assumption, to be related to the lecture's real-ocean pumps in B4.

B2. Focus on the deep-box balance

For the next derivation, use **deep-box inputs minus outputs**. Let m_d be the fixed deep-water mass in kg. Multiplying its DIC concentration tendency by m_d gives the inventory tendency in mol C/yr:

$$m_d \frac{dDIC_d(t)}{dt} = J_{mix,down}(t) + J_{pump}(t) - J_{mix,up}(t).$$

The pump enters the deep box; mixing carries carbon both downwards and upwards. The **net upward mixing** shown in the plot is the difference of those two mixing arrows:

$$J_{mix}(t) = J_{mix,up}(t) - J_{mix,down}(t) = Q\rho[DIC_d(t) - DIC_s(t)].$$

▼ Reference — complete carbon and TA budgets

Let m_s be surface-water mass and C_{atm} atmospheric carbon in mol C. The other two carbon budgets are

$$m_s \frac{dDIC_s(t)}{dt} = J_{gas,in}(t) + J_{mix,up}(t) - J_{gas,out}(t) - J_{mix,down}(t) - J_{pump}(t),$$

$$\frac{dC_{atm}(t)}{dt} = J_{gas,out}(t) - J_{gas,in}(t).$$

Adding all three budgets cancels every internal carbon transfer. The pump leaves the surface and enters the deep box at the same rate. For TA, the surface budget is $J_{mix,up}^{(TA)}(t) - J_{mix,down}^{(TA)}(t)$ and the deep budget is its opposite; gas exchange and this pump have no TA terms.

B3. Exercise: derive the expression before coding

calibration-first: use the reference DIC ratio to infer k . Agreement with that ratio will check implementation, rather than independently predict the gradient.

Question — derive and implement the pump

1. At a stationary state, what must the deep-box tendency be? Use that condition to relate pump and mixing fluxes, writing stationary concentrations with stars.
2. Rearrange that relationship to obtain an expression for k in terms of Q , ρ , DIC_s^* and DIC_d^* . Determine the units of k when DIC is expressed in mol/kg.
3. Evaluate your expression with the observed reference DIC values 2040 and 2250 $\mu\text{mol/kg}$ and the independently selected transport of 20 Sv.
4. Complete **02.3** in `build_layers`: choose the pump endpoints, `pump_type` and scale. Explain which law from A3 implements the assumption in B1, then rerun the definition cell. Enter your expression for k below and run the supplied comparison at the same initial total carbon and TA.

The supplied `mixing_mass_transport()` returns $Q\rho$ in kg/yr using ESBMTK density and the prescribed 20 Sv. Use the same DIC units in your concentration ratio. The flux plot uses Tmol C/yr, where 1 Tmol = 10^{12} mol.

Instructor answer

Zero deep accumulation first gives inputs equal to outputs, $J_{mix,down}^* + J_{pump}^* = J_{mix,up}^*$. Substituting the flux laws gives

$$Q\rho(DIC_d^* - DIC_s^*) = kDIC_s^*.$$

Dividing by the positive stationary surface concentration gives

$$k = Q\rho \left(\frac{DIC_d^*}{DIC_s^*} - 1 \right).$$

k has units kg/yr so that $kDIC_s(t)$ is mol/yr. Convert 20 Sv to m³/yr before multiplying by ESBMTK density. With the defaults, $k \simeq 6.6644 \times 10^{16}$ kg/yr. In 02.3, use the concentration-dependent law with surface DIC as source, deep DIC as sink and k as its scale.

Instructor note. The observed DIC ratio constrains $k/(Q\rho)$, rather than k and Q individually. The inverse-time coefficient for the surface concentration tendency would be k/m_s ; neither point is a separate task.

```
# Choose and explain: complete marked choices; surrounding machinery is supplied.
# BEGIN SOLUTION
k = mixing_mass_transport() * (config.target_deep_dic_umol_kg / config.target_dic_umol_kg - 1)
# END SOLUTION
print('Fitted k (kg/yr):', k)
```

Fitted k (kg/yr): 6.6644448512018056e+16

Run the supplied pump comparison

After updating **02.3**, rerun the `build_layers` definition and the cell below. Both runs have the same total carbon and TA; only the pump changes. Use the figure to follow the atmospheric CO₂ response and see downward pumping balance net upward mixing. The supplied checks verify conservation, the fitted DIC ratio and stationary flux balance.

```
# Understand and run: trace this supplied step and interpret its evidence.
pump_on = build_layers(k)
run_model(pump_on)
audit(pump_on)
np.testing.assert_allclose(pump_on.Deep.DIC.c[-1] / pump_on.Surface.DIC.c[-1],
                           config.target_deep_dic_umol_kg / config.target_dic_umol_kg, rtol=1e-4)
Jmix = mixing_mass_transport() * (pump_on.Deep.DIC.c - pump_on.Surface.DIC.c)
Jpump = k * pump_on.Surface.DIC.c
np.testing.assert_allclose(Jmix[-1], Jpump[-1], rtol=1e-4)
plot_pump_comparison(pump_off, pump_on, Jmix, Jpump)
print('Checks passed: carbon/TA conserved; fitted DIC ratio and pump-mixing balance recovered.')
```

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G.Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it
under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

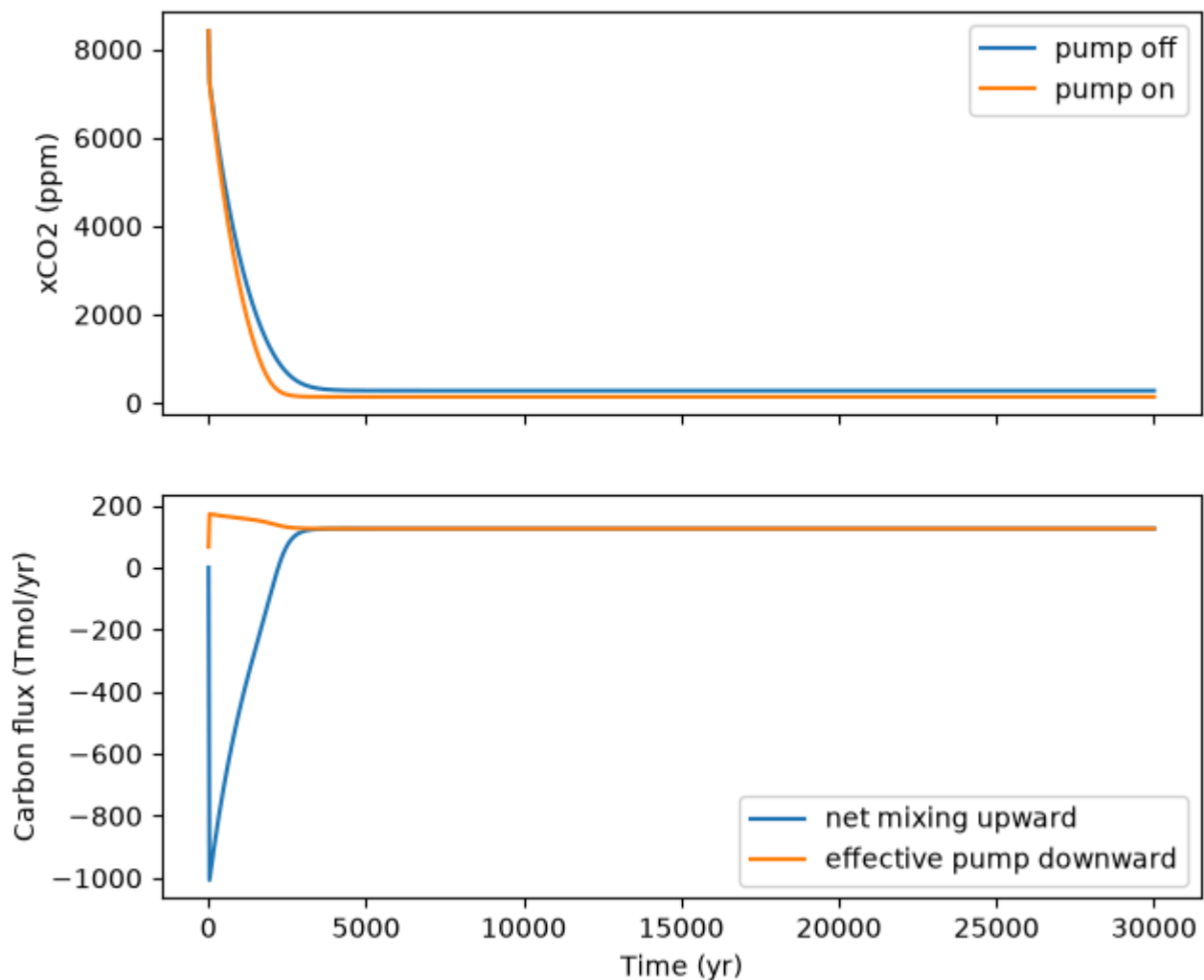
Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

status=0

message=The solver successfully reached the end of the integration interval.

Execution took 1.69 CPU seconds, wall time = 1.72 seconds

There were 82 warnings, check M.log



Checks passed: carbon/TA conserved; fitted DIC ratio and pump-mixing balance recovered.

B4. Relate the effective pump to the real ocean

With the same total carbon, the pump transfers carbon towards the deep ocean and lowers atmospheric CO₂.

Primary production fixes carbon into organic matter; **export** carries organic carbon out of the surface layer; **remineralization** converts organic carbon back to dissolved inorganic carbon.

Question — relate the model to the real ocean

Which real-ocean pump does this DIC-only transfer most closely resemble? Sketch production → export → remineralization beside its DIC arrow. Which steps are collapsed into that arrow? Does it represent primary production, export reaching the deep box, or permanent burial? Name one omitted control.

Instructor answer

It most closely resembles the **soft-tissue biological pump**: carbon removed near the surface returns to DIC at depth through remineralization (breakdown of organic matter). One DIC arrow replaces production, sinking and recycling; it represents effective export reaching the deep box and remineralization there, not total primary production or permanent burial. Surface recycling and a separate particle inventory are omitted; mixing can return this carbon to the surface. Nutrients, light and the depth distribution of recycling are examples of omitted controls. Uniform temperature omits the thermal mechanism of the solubility pump, and this transfer has no TA change associated with the carbonate pump. Fitting the full DIC ratio does not isolate a measured soft-tissue contribution.

C. Calculate a carbon input and verify the forced run

We chose **k** to reproduce the reference DIC ratio. This determines the relative concentrations, but the total carbon inventory also matters. Part B kept the original 01 inventory. How much additional carbon is needed to reach the **full reference state**: 280 ppm in the atmosphere, 2040 $\mu\text{mol/kg}$ surface DIC and 2250 $\mu\text{mol/kg}$ deep DIC?

At that reference state, the prepared geometry gives the whole-reservoir **inventory ratio**

$r_{\text{ocn/atm}} = C_{\text{ocn}}/C_{\text{atm}} = 62.4$: 62.4 moles of ocean carbon per mole of atmospheric carbon. This is a ratio of **existing carbon stocks** at the reference state, not the fraction of a future carbon addition absorbed by the ocean.

Notation: $r_{\text{ocn/atm}}$ is the ocean/atmosphere carbon inventory ratio. Reserve **R** for the Revelle factor; **F** denotes the lecture's seawater equilibrium capacity.

▼ Optional reference — connection to lecture slides 18–21

The lecture's **F** is seawater equilibrium carbon capacity. In the dry-air mole-fraction convention used here, $F = \text{DIC}_s^*/x_{\text{CO}_2}^* \simeq 7.3$ mol air/kg seawater. For the uniform ocean in 01, $r_{\text{ocn/atm},0} = Fm_{\text{ocn}}/N_{\text{atm}} \simeq 57$, where m_{ocn} is ocean water mass and N_{atm} is atmospheric moles of air. In 02 the deep DIC excess increases the inventory ratio to 62.4 at the reference state. Keep **F** and $r_{\text{ocn/atm}}$ distinct; use the actual inventory below rather than the rounded lecture ratio.

C1. Derive and calculate the required addition

Question — derive the carbon addition

Before running any forcing, use the following inputs to calculate how much extra carbon would make that reference pumped state possible:

Input	Where to find it
Reference pumped inventory ratio, $r_{\text{ocn/atm}} = 62.4$	<code>config.pumped_ocean_atmosphere_ratio</code>
Reference atmospheric xCO ₂ , 280 ppm	<code>config.target_xco2_ppm</code>
Total atmospheric gas inventory (mol)	<code>config.atmosphere_mol</code>
Existing atmosphere-plus-ocean carbon (mol C)	<code>config.total_carbon_mol</code>

1. Convert the reference atmospheric mole fraction into moles of atmospheric carbon.
2. Use 62.4 to calculate the ocean carbon and then the combined target inventory.
3. Subtract the existing total carbon to find the extra carbon to inject. Use the actual inventory; do not substitute a rounded baseline ratio of 57.
4. Check your answer against the extra carbon held in the deep box relative to a uniform ocean at the reference surface DIC. Explain why the two expressions agree.
5. In **02.4**, assign the atmospheric reference inventory to `C_atm_280`, the combined target to `target_total_carbon`, and the addition in **mol C** to `extra_carbon_mol`. The supplied forcing uses the addition as its `mass`.

Record your expected endpoint before running the model. Inventory output uses Pmol C, where 1 Pmol = 10^{15} mol.

Instructor answer

Let $C_{\text{atm},280} = N_{\text{atm}}(280 \times 10^{-6})$ and let C_0 denote the existing combined inventory. The required addition is

$$\Delta C = (1 + 62.4)C_{\text{atm},280} - C_0.$$

Equivalently, calculate the actual unpumped ocean/atmosphere ratio $r_{\text{ocn/atm},0} = (C_0 - C_{\text{atm},280})/C_{\text{atm},280}$ and use $\Delta C = (62.4 - r_{\text{ocn/atm},0})C_{\text{atm},280}$. With the default inputs, $r_{\text{ocn/atm},0} \simeq 56.9998256$; using exactly 57 introduces a small rounding mismatch.

Because the prepared volumes sum to the 01 ocean volume, subtracting the two reference ocean inventories also gives

$$\Delta C = m_d(DIC_d^* - DIC_s^*).$$

These expressions yield about 267.6326 Pmol C. The atmospheric reference inventory is included in both total inventories and cancels in the difference.

```
# Choose and explain: complete marked choices; surrounding machinery is supplied.
# Exercise 02.4: evaluate your derived carbon addition in mol C.
# Assign C_atm_280, target_total_carbon and extra_carbon_mol.
# BEGIN SOLUTION
C_atm_280 = config.atmosphere_mol * config.target_xco2_ppm * 1e-6
target_total_carbon = (1 + config.pumped_ocean_atmosphere_ratio) * C_atm_280
extra_carbon_mol = target_total_carbon - config.total_carbon_mol
```

```
# END SOLUTION
print('Carbon to inject (Pmol C):', extra_carbon_mol / 1e15)
```

Carbon to inject (Pmol C): 267.63264476779057

```
# BEGIN SOLUTION
# Supplied independent algebraic check of the same prepared reference inventory.
extra_carbon_from_boxes = box_mass_kg(pump_on.Deep) * (
    config.target_deep_dic_umol_kg - config.target_dic_umol_kg) * 1e-6
np.testing.assert_allclose(extra_carbon_mol, extra_carbon_from_boxes, rtol=1e-12)
print('Actual baseline ocean/atmosphere ratio:',
      (config.total_carbon_mol - C_atm_280) / C_atm_280)
# END SOLUTION
```

Actual baseline ocean/atmosphere ratio: 56.99982556965717

C2. Run the supplied forcing and budget checks

A **restart** uses the Part B final concentrations as the initial state of a new run. The **control** receives no input; the **forced run** receives your extra carbon through the atmosphere.

signal prescribes the flux over time; its mass is the total carbon addition. source represents carbon outside the system, connected to the atmospheric reservoir by Species2Species.

The supplied clock helper resolves the pulse automatically. clock is a dictionary of timing settings; **clock passes them to both model constructors. If you change pulse_duration, use positive whole-year durations and rerun this whole cell to build fresh models.

Read the final check summary: **input mass** → **carbon/TA budgets** → **reference endpoint**. The figure shows the input and the atmospheric response.

▼ Optional — changing pulse duration and numerical details

At fixed added mass, a longer square pulse has a smaller flux rate. Duration changes the transient; after sufficient relaxation, the final equilibrium depends on the total addition. Try, for example, '100 yr', '333 yr' or '1 kyr'. Keep the pulse inside the run and leave time after it to settle; increase run_stop if the endpoint checks show that it has not settled yet.

finite_pulse_clock aligns the pulse with the model grid and refines the grid for short pulses. The solver joins samples with straight lines, so the code checks both the sampled sum and the interpolated integral against your addition. The signal is zero at both simulation boundaries. The budget check compares total carbon with the initial inventory plus cumulative input, while TA stays constant. Endpoint checks compare atmospheric CO2 and both DIC concentrations with the reference state.

```
# Understand and run: trace this supplied step and interpret its evidence.
# Supplied forcing example: insert your calculated extra_carbon_mol above.
pulse_start = '1000 yr'
pulse_duration = '100 yr' # Change this value to explore another duration.
run_stop = '30 kyr'
clock = finite_pulse_clock(start=pulse_start, duration=pulse_duration, stop=run_stop)
state = {b.name: (b.DIC.c[-1] * 1e6, b.TA.c[-1] * 1e6) for b in pump_on.ocean_boxes}
forced = build_layers(k, state=state, **clock)
control = build_layers(k, state=state, **clock)
# Recompute atmospheric carbon by conservation from the pump-on restart.
np.testing.assert_allclose(forced.CO2_At.c[0], pump_on.CO2_At.c[-1], atol=1e-10)

signal = Signal(name='synthetic_carbon', species=forced.CO2, register=forced,
               start=pulse_start, duration=pulse_duration,
               mass=f'{extra_carbon_mol} mol', shape='square')
source = Source(name='external_carbon', species=forced.CO2)
connection = Species2Species(source=source, sink=forced.CO2_At,
```

```

        rate='0 mol/yr', signal=signal, id='synthetic_input')
    signal_time, signal_flux = forced.time.copy(), signal.m.copy()
    sampled_mass = signal_flux.sum() * forced.dt
    continuous_mass = integrated_signal(signal_time, signal_flux, signal_time[-1])
    np.testing.assert_allclose([sampled_mass, continuous_mass], extra_carbon_mol, rtol=1e-12)
    for case in (control, forced):
        run_model(case)
    added = integrated_signal(signal_time, signal_flux, forced.time)
    audit(control)
    audit(forced, added)
    np.testing.assert_allclose(forced.CO2_At.c[-1] * 1e6, 280, atol=0.5)
    np.testing.assert_allclose(forced.Surface.DIC.c[-1] * 1e6, 2040, atol=0.2)
    np.testing.assert_allclose(forced.Deep.DIC.c[-1] * 1e6, 2250, atol=0.2)
    plot_synthetic_forcing(signal_time, signal_flux, forced, control)
    print('Checks passed: input mass, carbon/TA budgets and full reference endpoint.')

```

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G.Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it
under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

ESBMTK 0.14.3.1.post0

Copyright (C) 2020 - 2026 Ulrich G.Wortmann

This program comes with ABSOLUTELY NO WARRANTY

This is free software, and you are welcome to redistribute it
under certain conditions; See the LICENSE file for details.

If you use ESBMTK for your research, please cite:

Wortmann et al. 2025, <https://doi.org/10.5194/gmd-18-1155-2025>

status=0

message=The solver successfully reached the end of the integration interval.

Execution took 9.14 CPU seconds, wall time = 9.18 seconds

status=0

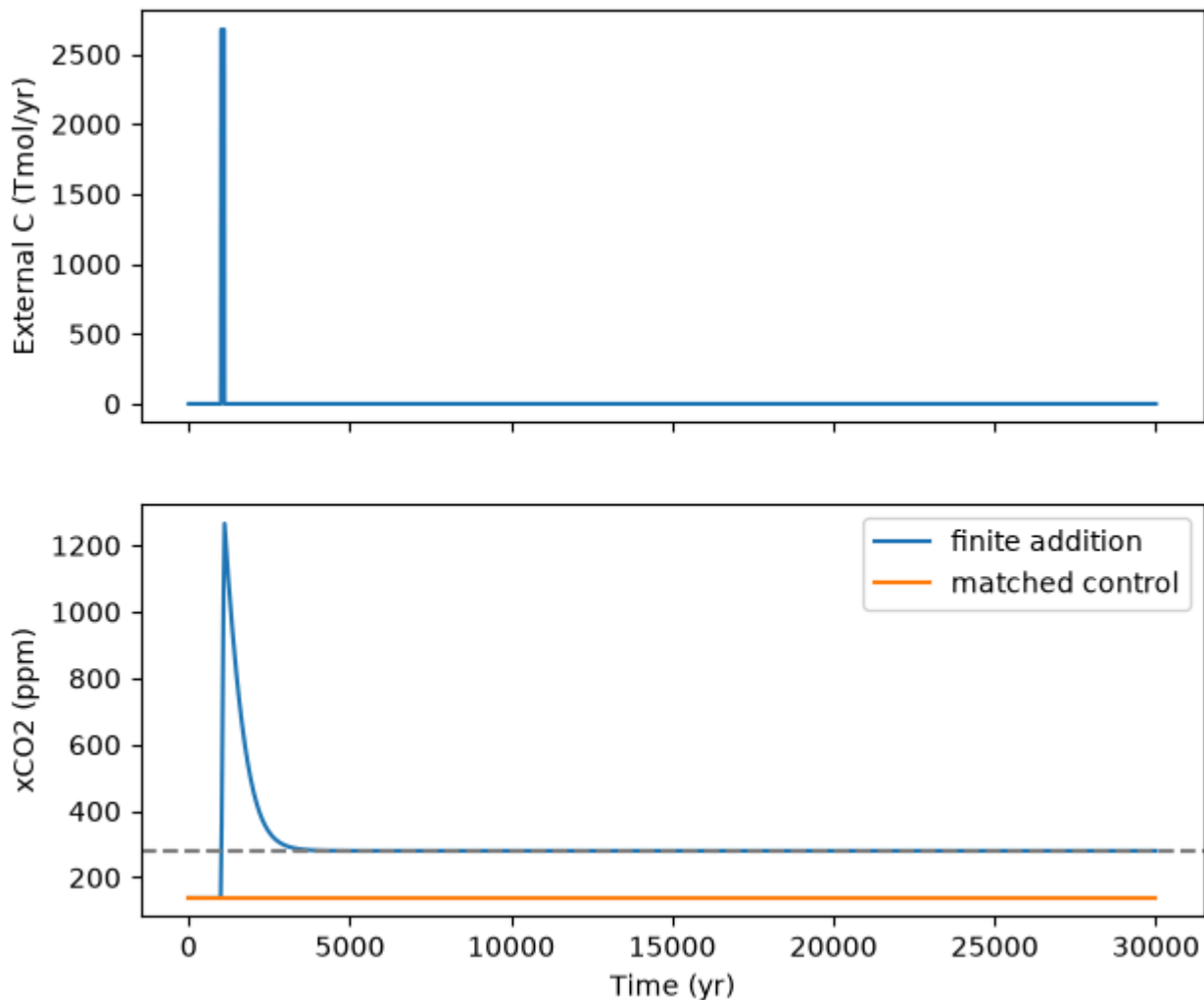
message=The solver successfully reached the end of the integration interval.

Execution took 6.23 CPU seconds, wall time = 6.29 seconds

=====

There were 29 warnings, check M.log

=====



Checks passed: input mass, carbon/TA budgets and full reference endpoint.

C3. Explain what the forcing experiment establishes

Question — explain the forcing check

Does the final state agree with your prediction in C1? Explain why recovering 280 ppm checks the supplied forcing and carbon budget without independently validating the effective pump. Point to the external carbon-input connection and explain how it differs from the internal transfers in A/B.

Instructor answer

The total reference inventory is the sum of the target ocean and atmospheric carbon. Subtracting the existing combined inventory gives exactly the required external input. The prepared geometry, target-derived TA and inferred pump coefficient make this state internally consistent. Returning there verifies the forcing implementation and analytical carbon budget; it is not independent validation of the effective pump. A different addition generally gives a different atmospheric endpoint through nonlinear carbonate chemistry. The `synthetic_input` connection runs from an external source to atmospheric CO₂, increasing combined carbon. Mixing, pumping and air-sea exchange only redistribute carbon within that combined inventory.

If the reference state is not recovered, inspect signal units and integrated mass, conservation, equilibration and chemistry settings before changing inputs.

The native solver may flag pH changes between stored output points during these large transients. Use the explicit inventories, stationary flux balance and endpoint checks above; transient pH interpretation is outside this practical.

You have completed 02. Keep your two derivations, including units, and your explanations in this notebook; no separate submission is required. Continue to 03 for explicit circulation and biological/carbonate processes.